SUPPLY MANAGEMENT SYSTEM

Task Addendum

MIR-to-PR Linkage and Procurement Chain Timeline

Maps to FSD Addendum 21 v1.1 | PR status NEVER changed by disbursement

Version 1.1 | 2026

FSD source	FSD Addendum 21 v1.1 — MIR-to-PR Linkage and Procurement Chain Timeline
Total tasks	10
Total estimated dev days	25 days (~5 working weeks)
Critical priority tasks	4
Total unit test cases	59
Parts	Part A (MIR-to-PR Linkage) — 4 tasks; Part B (Timeline) — 6 tasks
Key rule	PR line status is NEVER modified by Part A. Only disbursed_qty and disbursed_mir_ids are written. No disbursement_status column exists.

Table of Contents

Table of Contents 2
1. FSD Traceability Summary 3
2. Detailed Task Register 4
3. Sequencing Notes 10

1. FSD Traceability Summary

Every task maps to FSD Addendum 21 v1.1. Part A tasks explicitly confirm PR line status is never modified.

Task ID	Part	Title	Days	FSD Ref	Priority
TL-001	A	Approved PR line search API and MIR line pr_line_id FK	2d	Sec 2.2	High
TL-002	A	MIR line Fetch PR button — React UI	2d	Sec 2.2	High
TL-003	A	PR line disbursement quantity tracking on MIR approval (no status change)	3d	Sec 2.3	Critical
TL-004	A	PR screen computed disbursement badge and drill-down panel	2d	Sec 2.4, 2.5	High
TL-005	B	Add trace_id column to all 6 document tables	2d	Sec 3.2, 3.3	Critical
TL-006	B	trace_id propagation logic across the procurement chain	3d	Sec 3.2	Critical
TL-007	B	DocumentTimelines table and ITimelineService implementation	3d	Sec 4, 6	Critical
TL-008	B	Wire 22 business events to AppendEventAsync via Hangfire	3d	Sec 5	High
TL-009	B	Timeline API endpoints	2d	Sec 7.1	High
TL-010	B	Timeline UI panel on all document detail screens	3d	Sec 7.2	High
TOTAL		10 tasks	25d		

2. Detailed Task Register

Part A — MIR-to-PR Line Linkage (FSD Sections 2.2-2.5)

Optional PR line reference on MIR lines, Fetch button, disbursement qty tracking (NOT status changes) on PR lines, computed visual badge.

TL-001	High	2d	Sec 2.2	Approved PR line search API and MIR line pr_line_id FK

Add the optional pr_line_id FK to MIRLines and build the API endpoint that the Fetch button calls to find approved PR lines with remaining undisbursed quantity for a given product.

Development tasks	Unit test cases
• Add nullable pr_line_id (FK to demand_schema.PurchaseRequisitionLines) to material_schema.MaterialIssueRequestDetail • EF Core migration: dotnet ef migrations add AddMirPrLineLink • GET /api/purchase-requisitions/search?productId={id}&status=APPROVED — returns approved PR lines matching the product, filtered to WHERE disbursed_qty < quantity (computed check, no status column involved) • Response DTO: prNumber, prTitle, lineDescription, requestedQty, remainingUndisbursedQty (quantity minus disbursed_qty), prLineId • POST and PATCH for MIR accept optional pr_line_id — no validation beyond FK existence and PR status = APPROVED • pr_line_id clearable while MIR status = DRAFT, locked once submitted	• GET search with productId for Lenovo Laptop returns only APPROVED PRs containing that product • GET search excludes PR lines where disbursed_qty >= quantity (fully disbursed) • Saving an MIR line with a valid pr_line_id succeeds • Saving an MIR line with pr_line_id pointing to a non-APPROVED PR returns 400 • Saving an MIR line with pr_line_id = null succeeds (field is optional) • Attempting to change pr_line_id on a submitted (non-DRAFT) MIR returns 422

Acceptance criteria	
	• Fetch endpoint filters by product and computed undisbursed quantity, no disbursement_status column involved • pr_line_id truly optional and lockable after submission

TL-002	High	2d	Sec 2.2	MIR line Fetch PR button — React UI

Build the Fetch button and dropdown on the MIR line form that calls the search API, displays matching PRs, and stores the selected pr_line_id.

Development tasks	Unit test cases
• React: after Product is selected on the MIR line form, render Link to Purchase Requisition field with Fetch button • Fetch calls GET /api/purchase-requisitions/search?productId={selectedProductId}&status=APPROVED • Dropdown shows: PR-{number} — {title} — {lineDescription} — Requested: {qty} — Remaining: {remainingUndisbursedQty} • Selecting stores pr_line_id and shows PR number as a read-only badge	• Fetch calls correct endpoint with selected product ID • Selecting a PR stores pr_line_id correctly • Clearing removes pr_line_id • Fetch button disabled when MIR not DRAFT

• Clearing sets pr_line_id to null (DRAFT only) • Zero results shows: No approved PRs with remaining undisbursed quantity found for this product	• Zero results shows message without breaking the form

Acceptance criteria	• Fetch-and-select workflow intuitive and correctly wires pr_line_id • User never forced to link a PR

TL-003	Critical	3d	Sec 2.3	PR line disbursement quantity tracking on MIR approval (no status change)

Add disbursed_qty and disbursed_mir_ids to PRLines. On MIR approval, update these tracking fields ONLY — the PR line status is NEVER touched, read, or modified by this process.

Development tasks	Unit test cases
• Add disbursed_qty (Decimal, default 0) and disbursed_mir_ids (NVARCHAR(MAX) JSON array, default empty) to PurchaseRequisitionLines in DemandDbContext — NO disbursement_status column • EF Core migration: dotnet ef migrations add AddPrLineDisbursementTracking — default existing rows to disbursed_qty = 0 • Subscribe to DocumentApprovedEvent (MediatR) for interfaceCode=MIR: for each MIR line where pr_line_id IS NOT NULL, update the linked PR line's tracking fields only • Update logic: PRLine.disbursed_qty += MIRLine.approved_qty; append MIR.request_id to disbursed_mir_ids JSON array • Explicitly confirm: the handler NEVER reads, writes, or references PRLine.line_status or PRLine.status — those fields are completely outside the scope of this handler • Handle concurrent MIR approvals for the same PR line using serializable transaction or optimistic concurrency on disbursed_qty	• MIR approval with one line linked to PR line (qty 100, approved_qty 60): PRLine.disbursed_qty = 60 — PRLine.line_status is UNCHANGED from its pre-approval value • Second MIR approved with approved_qty 40: disbursed_qty = 100 — PRLine.line_status is STILL unchanged • MIR approval with pr_line_id = null on all lines does not touch any PR line field at all • disbursed_mir_ids contains both MIR IDs after both approvals • Concurrent approval of two MIRs against the same PR line: both succeed, disbursed_qty correct, no lost update • PR line with zero linked MIRs has disbursed_qty = 0 and its status remains exactly as it was at approval time — verified as a non-regression test

Acceptance criteria	• Only disbursed_qty and disbursed_mir_ids are ever written — PR line status is provably untouched • Concurrent safety verified • No disbursement_status column exists anywhere in the schema

TL-004	High	2d	Sec 2.4, 2.5	PR screen computed disbursement badge and drill-down panel

Add a computed visual badge per PR line on the PR detail screen (derived client-side from disbursed_qty vs quantity, not from any stored status), with a clickable drill-down panel listing linked MIRs.

Development tasks	Unit test cases

• React: PR detail screen, each PR line row shows its existing line_status exactly as before — no change to the status display • When disbursed_qty > 0, render an ADDITIONAL badge beside (not replacing) the existing status: amber showing X / Y disbursed when 0 < disbursed_qty < quantity; green Fully Disbursed when disbursed_qty >= quantity • When disbursed_qty = 0, render NO disbursement badge at all — the line looks identical to before this feature • Badge is computed entirely in the React component from the API response fields (disbursed_qty, quantity) — no database status column is read for this • Badge is clickable — opens slide-over panel listing all linked MIRs via disbursed_mir_ids • GET /api/purchase-requisitions/{prId}/lines/{lineId}/disbursements — returns linked MIRs: mir_number, approved_date, approved_qty, project/department	• PR line with disbursed_qty = 0 shows NO disbursement badge — only the existing line_status is visible • PR line with disbursed_qty = 60 and quantity = 100 shows amber badge 60 / 100 disbursed BESIDE the unchanged APPROVED status • PR line with disbursed_qty >= quantity shows green Fully Disbursed badge BESIDE the unchanged APPROVED status • Clicking the badge opens panel with correct linked MIRs • Deep link from panel to MIR detail screen works correctly • PR line's existing line_status badge is NEVER hidden, replaced, or visually altered by the disbursement badge

Acceptance criteria	• Badge is purely computed from disbursed_qty vs quantity, no stored status involved • Existing PR line status display completely unaffected • Drill-down panel shows accurate linked MIRs

Part B — Procurement Chain Timeline (FSD Sections 3-7)

trace_id propagation across all 6 document types, DocumentTimelines table, ITimelineService, 22 business events, Timeline UI panel.

TL-005	Critical	2d	Sec 3.2, 3.3	Add trace_id column to all 6 document tables

Add trace_id (UNIQUEIDENTIFIER NOT NULL) to PurchaseRequisitions, Quotations, PurchaseOrders, GRNs, Invoices, and MaterialIssueRequest.

Development tasks	Unit test cases
• Add trace_id (Guid, required) to 6 entities across 5 DbContexts • Run 6 EF Core migrations: dotnet ef migrations add AddTraceId_{ContextName} • Each migration sets DEFAULT NEWSEQUENTIALID() for existing rows • Add non-clustered index on trace_id in each table	• After migration, every existing row has a non-null unique trace_id • No two existing rows share the same trace_id • New PR via API has a system-generated trace_id • trace_id present in GET response for all 6 document types

Acceptance criteria	
	• All 6 tables have trace_id with no nulls • Index present on each table

TL-006	Critical	3d	Sec 3.2	trace_id propagation logic across the procurement chain

Implement propagation rules: PR generates new trace_id; downstream documents inherit from their source; standalone documents generate their own.

Development tasks	Unit test cases
• PR creation: trace_id = Guid.NewGuid() • Quotation from PR: copy PR.trace_id; standalone: generate new • PO from PR: copy PR.trace_id; PO from Quotation: copy Quotation.trace_id; manual: generate new • GRN: copy PO.trace_id; Invoice: copy PO.trace_id • MIR: if any line has pr_line_id, copy that PR's trace_id (first linked line wins); else generate new • Log warning if MIR lines reference different trace chains • trace_id immutable after creation	• PR gets trace_id A; Quotation from PR gets A; PO from Quotation gets A; GRN gets A; Invoice gets A — full chain confirmed • MIR linked to that PR gets trace_id A • Manual PO gets unique trace_id B • Standalone Quotation gets unique trace_id C • MIR with no linkage gets unique trace_id D • Editing PR does not change trace_id • MIR with lines from two different chains logs warning, saves with first PR's trace_id • Second MIR against same PR also inherits trace_id A

Acceptance criteria	
	• Full chain shares one trace_id • Standalone documents isolated • trace_id immutable

TL-007	Critical	3d	Sec 4, 6	DocumentTimelines table and ITimelineService implementation

Create DocumentTimelines entity and implement concurrency-safe AppendEventAsync, GetTimelineAsync, and ResolveTraceIdAsync.

Development tasks	Unit test cases
• DocumentTimelines entity: timeline_id, trace_id UQ, events NVARCHAR(MAX), chain_root_type, chain_root_ref, first_event_at, last_event_at, created_at • EF Core migration: dotnet ef migrations add AddDocumentTimelines • AppendEventAsync: UPDLOCK, read JSON, deserialize, append, serialize, write back; upsert on first event • GetTimelineAsync: query by trace_id, return events ordered ascending • ResolveTraceIdAsync: dispatch on interfaceCode to correct DbContext • Concurrency test: two simultaneous appends both succeed	• Append on new trace_id creates row with single-element array • Append on existing trace_id extends the array • Two concurrent appends both succeed, array contains both • GetTimelineAsync returns events in ascending order • ResolveTraceIdAsync for a PO returns correct trace_id • Resolve for non-existent document returns null • last_event_at updates on every append; first_event_at never changes

Acceptance criteria	
	• Concurrency-safe with no lost updates • Resolve dispatches across 6 DbContexts • JSON integrity maintained

TL-008	High	3d	Sec 5	Wire 22 business events to AppendEventAsync via Hangfire

Add BackgroundJob.Enqueue calls for every business event listed in FSD Section 5 across all document services.

Development tasks	Unit test cases
• PurchaseRequisitionService: 4 events (Created, Submitted, Approved, Rejected) • QuotationService: 3 events (Created from PR, Sent, Awarded) • PurchaseOrderService: 5 events (Created, Submitted, Approved, Sent, Amended) • GrnService: 3 events (Created, Approved, Rejected) • InvoiceService: 3 events (Received, Approved, Paid) • MaterialIssueRequestService: 3 events (Created, Approved, Issued) • MaterialReturnService: 1 event (Received) • All via Hangfire — never inline within business transaction	• PR creation enqueues Hangfire job that appends event • PO approval event includes approver and tier • GRN approval includes qty_accepted/rejected • Invoice includes amount and match status • MIR issued includes item, qty, recipient • Simulated AppendEventAsync failure does NOT roll back the business action

Acceptance criteria	
	• All 22 events wired with correct descriptions • Timeline writes non-blocking • Hangfire retry handles transient failures

TL-009	High	2d	Sec 7.1	Timeline API endpoints

Build the two GET endpoints for timeline retrieval — by trace_id and by document resolution.

Development tasks	Unit test cases
• GET /api/timeline/{traceId} — full timeline with ordered events • GET /api/timeline/by-document?interface={code}&documentId={id} — resolves trace_id then returns timeline; 404 if not found • Both available to any authenticated user • Response: traceId, chainRootType, chainRootRef, firstEventAt, lastEventAt, events[], totalEventCount	• GET by traceId returns all events chronologically • GET by-document for a PO returns the full chain, not just PO events • Non-existent document returns 404 • Unauthenticated returns 401 • totalEventCount matches array length

Acceptance criteria	
	• Both paths return complete chain • Resolution works for all 6 types

TL-010	High	3d	Sec 7.2	Timeline UI panel on all document detail screens

Build reusable slide-over Timeline panel and integrate into PR, Quotation, PO, GRN, Invoice, and MIR detail screens.

Development tasks	Unit test cases
• React: TimelinePanel component accepting traceId prop • Coloured dots by interface (PR=blue, Quotation=teal, PO=navy, GRN=green, Invoice=amber, MIR=purple), document_no as deep link, event_description, timestamp • Interface filter dropdown at top • View Timeline icon on 6 screens, each calls by-document endpoint • Slide-over on desktop, full-screen modal on mobile • Empty-state for pre-migration documents	• Panel on a PO shows full chain events (PR through MIR) • Filter correctly hides/shows by type • Deep links navigate correctly • Mobile renders as full-screen modal • Pre-migration document shows empty-state • Colours match 6 interface types

Acceptance criteria	
	• One reusable component on all 6 screens • Deep links work • Filter and ordering correct

3. Sequencing Notes

Parts A and B are independently deployable and can be developed in parallel.

Dependency	Reason
TL-001 before TL-002	Fetch UI needs the search API.
TL-001 before TL-003	Qty tracking reads pr_line_id from TL-001.
TL-003 before TL-004	Badge computes from disbursed_qty populated by TL-003.
TL-005 before TL-006	Propagation needs trace_id columns.
TL-006 before TL-007, TL-008	Service and wiring need trace_id values on documents.
TL-007 before TL-008	Event wiring calls AppendEventAsync.
TL-007, TL-009 before TL-010	UI calls API from TL-009 which calls service from TL-007.

— End of Document —